

Dynamic Graph Neural Policies for Multi-Robot Cooperative Navigation: Edge Features, Safety Filtering, and Density Limits

1st Xuelin Hu

*Liuzhou Railway Vocational
Technical College*
Liuzhou 545000, China
huxl@ltzy.edu.cn

2nd Xiaoqin Fu

*Liuzhou Railway Vocational
Technical College*
Liuzhou 545000, China
fuxiaoqin@ltzy.edu.cn

3rd Youjing Fu

Lanzhou University
Lanzhou 730000, China
fuyj2025@lzu.edu.cn

4th Jingchao Wang*

*Zhongyuan University
of Technology*
Zhengzhou 450007, China
static@zut.edu.cn

5th Pengming Hu

*Zhongyuan University
of Technology*
Zhengzhou 450007, China
hupengming@zut.edu.cn

6th Simeng Li

*Liuzhou Railway Vocational
Technical College*
Liuzhou 545000, China
lisimeng@ltzy.edu.cn

Abstract—Multi-robot navigation requires coordinated motion and collision avoidance under changing neighbor relations and limited communication. This paper presents a dynamic graph neural policy that represents robots as nodes and local communication links as edges. Five-dimensional relative-motion edge features encode relative position, velocity, and closing speed, while shared message passing produces local velocity commands for variable-size teams. The multi-scale policy uses mixed-expert imitation with separation, goal-progress, and velocity-consistency losses, and an external safety filter corrects approaching robot pairs during execution. Evaluation covers four map families and teams of 5–40 robots, with 30 unseen seeds per multi-scale configuration. Local message passing improves goal-reaching success over an edge-free policy of the same capacity. Safety filtering reduces close encounters and increases minimum inter-robot separation. In the tested 5–10-robot settings, the filtered policies achieve high collision-free success while retaining millisecond-level control time. These results support the use of local graph interaction and safety filtering for cooperative navigation at low to moderate team densities.

Index Terms—multi-robot systems, graph neural networks, cooperative navigation, collision avoidance, dynamic interaction graphs.

I. INTRODUCTION

Multi-robot cooperative navigation requires robots to reach assigned goals without colliding with one another or with static obstacles. As the number of robots increases, joint planning becomes more costly and local interactions become harder to coordinate. These interactions also change as neighbors enter and leave sensing range. Centralized search must account for the joint state space, whereas independent policies do not explicitly model the relationships among robots.

This work was supported by the Science and Technology Research Project of Henan Province (grant no. 262102211055) and the Key Scientific Research Project of Colleges and Universities in Henan Province (grant no. 27AQ520018).

*Corresponding author: Jingchao Wang.

Graph neural networks (GNNs) represent these relationships through robot nodes and communication edges. Message passing aggregates neighbor information, and parameter sharing allows the same policy to process different team sizes. Rebuilding edges from current positions updates the interaction structure as the robots move.

We examine how local graph communication affects goal-reaching success relative to a policy using only its own node features. We also evaluate how communication radius and safety filtering affect collision risk and control time. Comparisons use the same observations, action limits, and termination rules.

Our contributions are:

- a dynamic graph policy using five-dimensional edge features for normalized relative position, relative velocity, and closing speed, with shared message passing for variable-size robot teams;
- a training and execution scheme combining action imitation, one-step separation, goal progress, and smoothness losses with an optional external pairwise safety filter, whose effect is evaluated separately from the neural policy;
- a reproducible evaluation across four map families, 5–40 robots, and 30 unseen seeds, with classical and neural baselines and graph, radius, and safety ablations to measure the effects of robot density.

II. RELATED WORK

A. Classical and Rule-Based Planning

A* uses heuristic search to find paths on a discretized map [1]. In multi-robot settings, independently planned paths still require coordination to avoid conflicts. Velocity-obstacle methods identify velocities that would lead to collisions under

predicted relative motion [2]. Their geometric formulation supports local collision avoidance, with performance depending on the prediction horizon and the assumptions about neighboring agents' motion.

B. Learned Multi-Robot Collision Avoidance

Decentralized deep reinforcement learning maps local observations to motion commands, reducing the need for joint planning [3], [4]. Curriculum learning introduces increasingly difficult constrained passages [5]; physics-informed priors guide policy learning [6]; and semi-supervised methods reduce labeling requirements [7]. Related studies address communication and formation constraints in vessel navigation [8], [9].

C. Graph Representations for Robot Teams

Diao *et al.* encode spatial relations with a GNN for path planning [10]. He *et al.* combine graph attention with hierarchical motion planning to weight neighboring agents [11]. Goarin and Loiano use a GNN for decentralized goal assignment [12], while Yang *et al.* combine a GNN with PPO for multi-UAV planning [13]. A recent survey reviews rule-based and learned coordination methods for multi-robot collision avoidance [14].

This work combines a graph policy for local motion with a separate filter for approaching robot pairs. Graph and filter ablations distinguish the effects of learned interaction from those of action correction. Tests across four map families and five team sizes assess how these effects change with robot density.

III. PROBLEM FORMULATION

Let $p_i, v_i, g_i \in \mathbb{R}^2$ denote the position, current velocity, and goal of robot $i \in \mathcal{V} = \{1, \dots, N\}$. At each control step, the graph connects robot j to receiver i when

$$A_{ij} = \mathbf{1}\{0 < \|p_i - p_j\|_2 \leq R_c\}. \quad (1)$$

This excludes self-loops and pairs at exactly coincident positions, as in the implementation. Let $s = [W, H] = [16, 12]$ m and $v_{\max} = 1.4$ m/s. The eight-dimensional node feature is

$$x_i = [(g_i - p_i) \oslash s, v_i/v_{\max}, n_i^{\text{obs}}, \eta_i, 1], \quad (2)$$

where $\oslash$ denotes componentwise division, n_i^{obs} points from the nearest obstacle point toward the robot, and $\eta_i = \text{clip}(d_i^{\text{obs}}/4, 0, 1)$. For a zero obstacle displacement, this direction is zero. The code initializes obstacle distance to 9 m and direction to zero when no nearer obstacle is found.

For edge (j, i) , define $r_{ij} = p_i - p_j$, $\Delta v_{ij} = v_i - v_j$, and $n_{ij} = r_{ij} / \max(\|r_{ij}\|_2, \epsilon)$, with $\epsilon = 10^{-6}$. The edge tensor is

$$\begin{aligned} c_{ij} &= -\Delta v_{ij}^\top n_{ij}, \\ e_{ij} &= A_{ij} [r_{ij} \oslash s, \Delta v_{ij}/v_{\max}, c_{ij}/v_{\max}] \in \mathbb{R}^5. \end{aligned} \quad (3)$$

Positive c_{ij} indicates approach. Non-edge entries are zero; distance and communication indicators are not separate edge-feature components.

The executed velocity u_i satisfies $\|u_i\|_2 \leq v_{\max}$. The simulator updates $p_i^{t+1} = \Pi_\Omega(p_i^t + \Delta t u_i^t)$, where $\Delta t = 0.08$ s and

Π_Ω clips each coordinate to $\Omega = [-7.5, 7.5] \times [-5.5, 5.5]$ m. Safety-distance violations use $d_s = 0.72$ m; physical collisions use 0.56 m, the robot diameter. An event is counted on entry into the corresponding threshold, so a sustained violation counts once. Task success requires final mean goal error below 0.65 m. Strict success additionally requires no physical collision during the episode.

IV. METHOD

A. System Overview

Fig. 1 shows the inference loop for the edge-feature GNN. Robot observations form node features and a radius graph. A node encoder and a shared mean-edge encoder supply three message-passing layers, each performing aggregation and node update. The shared head predicts velocity; optional pairwise correction precedes norm clipping. Executed motion provides the next observation. Network inputs contain no candidate-action list.

B. Baselines

The baselines include A* grid tracking, an ORCA-style local repulsion controller, and an RVO-style controller that samples a finite lattice of headings and speeds. The RVO-style controller scores candidate velocities by predicted neighbor clearance, obstacle occupancy, and goal progress. It is implemented for this benchmark and is not a formally verified RVO reference implementation. The independent MLP shares parameters across robots but uses only node features, without the adjacency matrix.

C. Message-Passing Policy

The node encoder gives $h_i^{(0)} = \text{ReLU}(W_x x_i + b_x)$. Let $D_i = \max(1, \sum_j A_{ij})$. Each layer first averages neighboring states and edge features:

$$\bar{h}_i^{(l)} = \frac{\sum_j A_{ij} h_j^{(l)}}{D_i}, \quad \bar{e}_i = \frac{\sum_j A_{ij} e_{ij}}{D_i}. \quad (4)$$

The shared edge encoder is applied to the mean edge feature before the layer-specific message transform and node update:

$$\begin{aligned} z_i &= W_e \bar{e}_i + b_e, \\ m_i^{(l)} &= \text{ReLU}\left(W_m^{(l)} (\bar{h}_i^{(l)} + z_i) + b_m^{(l)}\right), \\ h_i^{(l+1)} &= \text{ReLU}\left(W_u^{(l)} [h_i^{(l)} \| m_i^{(l)}] + b_u^{(l)}\right). \end{aligned} \quad (5)$$

There are three layers of width 128. The edge encoder is shared across layers; message and update parameters differ between layers and are shared across robots. When no edge tensor is supplied, z_i is omitted. Supplying a zero edge tensor retains b_e , including for isolated nodes. The action head is Linear-ReLU-Linear and predicts a dimensionless vector $a_i = f_{\text{head}}(h_i^{(3)})$. The raw velocity is $u_i^{\text{raw}} = v_{\max} a_i$, without a final activation. Optional pairwise correction and norm clipping produce the executed velocity.

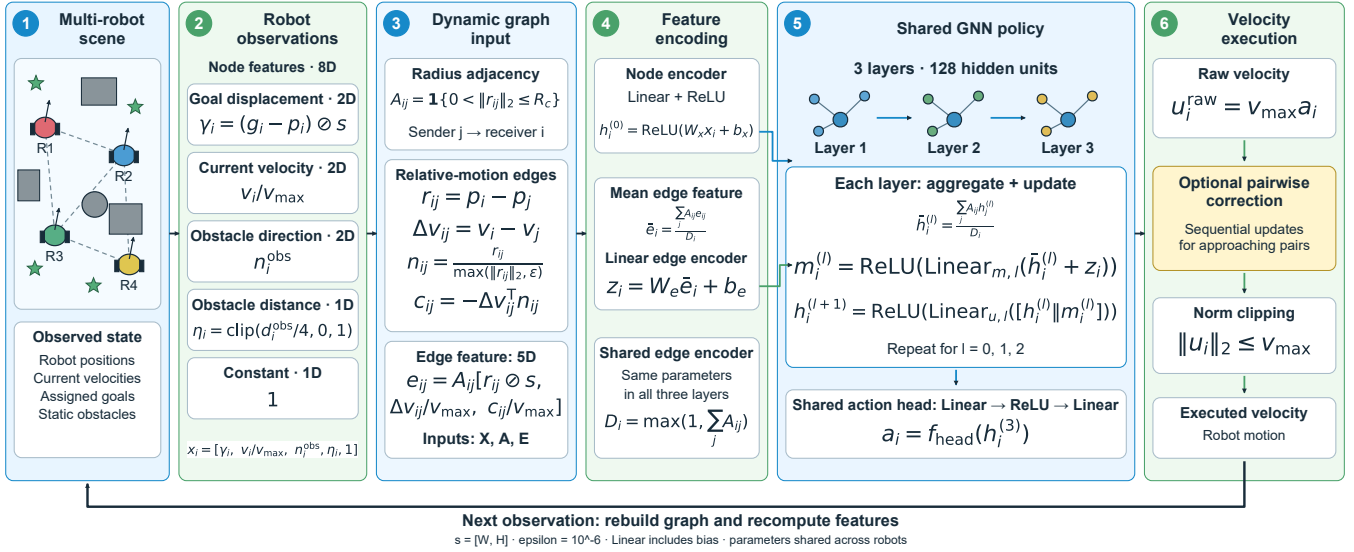


Fig. 1. Multi-robot navigation pipeline: robot scene and states, structured graph inputs, feature embedding, shared GNN inference, and action output with optional safety filtering. Executed motion closes the observation-action loop.

D. Attention Variants

Two attention models are evaluated with the same inputs, outputs, data, and test scenarios. The neighbor-attention GNN replaces mean aggregation with multi-head attention over queries, keys, and values, masked by the local adjacency. The graph transformer uses adjacency-masked multi-head self-attention, residual connections, and feed-forward blocks. Both models use 128 hidden units, four heads, and a shared policy head.

E. Training Objective

The training target is the normalized expert action $a_i^* = u_i^*/v_{\max}$. The trainer supports

$$\mathcal{L} = \mathcal{L}_{\text{act}} + \lambda_c \mathcal{L}_{\text{col}} + \lambda_p \mathcal{L}_{\text{prog}} + \lambda_s \mathcal{L}_{\text{smooth}}. \quad (6)$$

For an unpadded scene, $\mathcal{L}_{\text{act}} = N^{-1} \sum_{i,k} \text{SmoothL1}(a_{ik} - a_{ik}^*)$, with transition parameter 1 and both velocity components summed. Set $p'_i = p_i + \Delta t v_{\max} a_i$, $d'_{ij} = \|p'_i - p'_j\|_2$, and $\gamma_i = (g_i - p_i) \oslash s$. Then

$$\begin{aligned} \mathcal{L}_{\text{col}} &= \frac{\sum_{i \neq j} [d_s - d'_{ij}]_+^2}{N(N-1)}, \\ \gamma'_i &= \gamma_i - (\Delta t v_{\max} a_i) \oslash s, \\ \mathcal{L}_{\text{prog}} &= \frac{1}{N} \sum_i [\|\gamma'_i\|_2 - \|\gamma_i\|_2]_+, \\ \mathcal{L}_{\text{smooth}} &= \frac{1}{N} \sum_i \|a_i - v_i/v_{\max}\|_2^2. \end{aligned} \quad (7)$$

Here $[x]_+ = \max(x, 0)$. The progress term uses normalized goal displacement. The smoothness term compares the prediction with the current velocity in the recorded observation, not a previous network prediction. Training uses raw predictions before safety filtering, speed clipping, or boundary clipping. For padded minibatches, node sums are masked by q_{bi} and

divided by $\max(1, \sum_{b,i} q_{bi})$; pair sums use $q_{bi}q_{bj}$ for $i \neq j$ and the corresponding valid-pair count. Pairs never cross scene boundaries.

Saved records for all three multi-scale models specify $(\lambda_c, \lambda_p, \lambda_s) = (0.5, 0.5, 0.1)$, seed 7, and 30 epochs. Their mixed-expert datasets alternate ORCA-style, RVO-style, and safety-filtered ORCA-style labels, covering all four map families and 5–40 robots with padding to 40 nodes. Auxiliary weights default to zero; the joint objective is therefore specific to the recorded multi-scale runs, not an assumption about every checkpoint in the comparisons.

F. Safety Filter

Initialize $\tilde{u}_i = u_i^{\text{raw}}$. The filter visits each pair $i < j$ once, in index order, using the latest corrected velocities. Let $d_{ij} = \|r_{ij}\|_2$ and $b_{ij} = (\tilde{u}_i - \tilde{u}_j)^T n_{ij}$. If $d_{ij} < d_s + \rho$ and $b_{ij} < 0$, it applies

$$\begin{aligned} \alpha_{ij} &= \frac{1}{2} \{-b_{ij} + k_d [d_s + \rho - d_{ij}]_+\}, \\ \tilde{u}_i &\leftarrow \tilde{u}_i + \alpha_{ij} n_{ij}, \quad \tilde{u}_j \leftarrow \tilde{u}_j - \alpha_{ij} n_{ij}. \end{aligned} \quad (8)$$

The code uses a distance margin $\rho = 0.35$ m and implicit gain $k_d = 1 \text{ s}^{-1}$. Although the argument is named `horizon`, it is added to a distance, not used as a prediction time. After the pair sweep, the executed velocity is

$$u_i = \tilde{u}_i \min \left(1, \frac{v_{\max}}{\max(\|\tilde{u}_i\|_2, \epsilon)} \right). \quad (9)$$

With filtering disabled, the same clipping is applied to u_i^{raw} . This is a sequential pairwise correction, not an ORCA solver or a formal collision-free guarantee. It is used only during execution and when generating safety-filtered expert labels, not inside the training loss.

V. EXPERIMENTAL SETUP

Experiments span four map families—*crossing*, *open*, *random*, and *narrow*—and 5, 10, 20, 30, and 40 robots. Development data uses fixed seeds with 20 episodes of at most 160 steps each. The main evaluation covers $4 \times 3 \times 5 \times 10 = 600$ episodes over three robot counts, five methods, and ten fixed test seeds; the extended learned-model comparison covers 480 episodes for four learned policies; and the multi-scale test covers 1800 safety-filtered and 1800 unfiltered episodes over 30 unseen seeds (100–129). All methods share initial states, goals, obstacles, action limits, and termination rules. Data are stored as compressed NPZ archives holding features, adjacency, actions, positions, rewards, obstacles, layout, expert identifier, and step index.

Reported metrics are goal success rate, strict no-physical-collision success rate, safety-violation events, physical collision events, minimum inter-robot separation, path length, final goal error, and mean per-step control time. The recorded timer covers observation and policy computation, but excludes safety filtering and environment updates. Means, standard deviations, and 95% confidence intervals are computed over episodes, and violations at 0.72 m and physical collisions at 0.56 m are always reported in separate columns. Training used one 24 GB GPU with batch size 128.

VI. RESULTS

A. Main Comparison

Table I and Fig. 2 summarize the 600-episode main comparison. A* attains a task success rate of 1.000 at each team size. RVO-style achieves 0.975–1.000 success and the shortest paths, while its mean control time increases from 7.8 ms at 5 robots to 87.4 ms at 20 robots, approximately an 11-fold increase. ORCA-style has the lowest control time and the largest minimum separation at each team size. Among the learned policies in this comparison, the independent MLP achieves the higher success rates (0.725–0.875).

The mean-aggregation GNN reaches success rates of 0.625, 0.725, and 0.700 at 5, 10, and 20 robots. These rates are below those of the MLP, while the GNN has larger minimum inter-robot separation at each team size. The GNN checkpoint was trained only on 10-robot data; its 5- and 20-robot results therefore measure generalization across team sizes. The graph ablation below compares the same network with and without communication edges to isolate the effect of message passing.

B. Attention and Transformer Variants

Table II compares four learned policies over 480 episodes. Goal success is 0.817 for the MLP, 0.683 for the GNN, 0.558 for the graph transformer, and 0.342 for the neighbor-attention GNN. Their strict collision-free success rates are 0.017, 0.008, 0.058, and 0.025, respectively. The graph transformer has the highest strict success rate, with a mean control time of 1.365 ms compared with 0.856 ms for the GNN. Neighbor attention increases mean minimum separation to 0.259 m, but lowers goal success and takes 1.298 ms per step. These

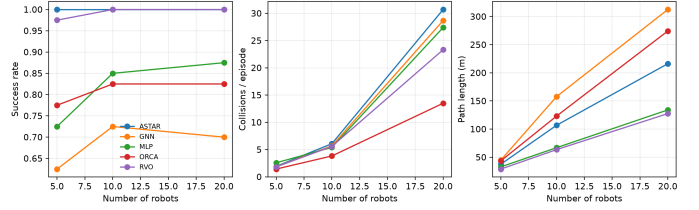


Fig. 2. Comparison of goal success, path length, and control time across methods and team sizes in the 600-episode main evaluation.

unfiltered policies remain insufficient for reliable collision-free navigation.

C. Graph Structure Ablation

The graph ablation compares local, global, and edge-free connectivity (Fig. 3a). At 10 robots, the local graph reaches 0.725 success compared with 0.425 for the edge-free policy of the same capacity, an increase of 30 percentage points. At 5 and 20 robots, the corresponding increases are 12.5 and 27.5 percentage points. Fully connecting the graph does not improve on local communication; at 10 robots, success is 0.625 for the global graph and 0.725 for the local graph. These results support local neighbor exchange for goal reaching, although collisions remain in both graph variants.

D. Safety Filter Ablation

The safety-filter ablation compares raw and corrected GNN actions over 60 episodes (Fig. 3b). At 10 robots, filtering reduces mean safety-violation events from 4.900 to 0.200 and increases mean minimum separation from 0.069 m to 0.794 m, while goal success decreases from 1.000 to 0.900. At 20 robots, violation events decrease from 27.500 to 6.900 and goal success from 0.900 to 0.700; mean minimum separation is 0.067 m. The correction improves spacing but can impede goal progress. Its limited effect at 20 robots motivates further study of safety constraints during training.

E. Communication Radius

Increasing the communication radius adds neighbors to each robot’s graph. At 10 robots, a radius of 2.0 m gives a mean degree of 0.463 and success of 0.200. Radii of 3.4 m and 5.5 m increase mean degree to 2.283 and 4.420, respectively, and both achieve success of 1.000. The global graph has mean degree 9.000 and success 0.900. At 5 robots, success increases from 0.100 at 2.0 m to 1.000 with the global graph. The results indicate that a small radius can restrict information exchange, while additional neighbors beyond an adequate local range do not consistently improve success. Control time ranges from about 0.8 to 1.5 ms across the tested settings, and physical collisions are not eliminated by global connectivity.

F. Multi-Scale Stress Test

Figs. 4 and 5 show scaling trends, map-specific success, and trajectories. Table III reports the 1800-episode safety-filtered test using 30 unseen seeds per configuration. At 5 and 10 robots, GNN strict collision-free success is 0.967 and 0.958, respectively, with no recorded physical collisions. The

TABLE I

MAIN COMPARISON OVER FOUR MAP FAMILIES AND TEN FIXED TEST SEEDS (40 EPISODES PER CELL). SUCC: GOAL SUCCESS. COLL: COLLISION EVENTS, COUNTING TRANSITIONS INTO A < 0.72 M SAFETY VIOLATION. SEP: MINIMUM INTER-ROBOT SEPARATION (M). T : MEAN PER-STEP CONTROL TIME (MS).

Method	5 robots				10 robots				20 robots			
	Succ	Coll	Sep	T	Succ	Coll	Sep	T	Succ	Coll	Sep	T
A*	1.000	1.925	0.280	0.512	1.000	6.075	0.098	1.205	1.000	30.700	0.035	2.493
ORCA-style	0.775	1.425	0.396	0.213	0.825	3.825	0.296	0.486	0.825	13.500	0.230	1.303
RVO-style	0.975	1.825	0.294	7.779	1.000	5.575	0.131	24.941	1.000	23.325	0.062	87.373
MLP	0.725	2.575	0.191	0.342	0.850	5.400	0.095	0.483	0.875	27.400	0.038	0.775
GNN	0.625	1.850	0.224	0.683	0.725	5.675	0.149	0.828	0.700	28.675	0.041	1.129

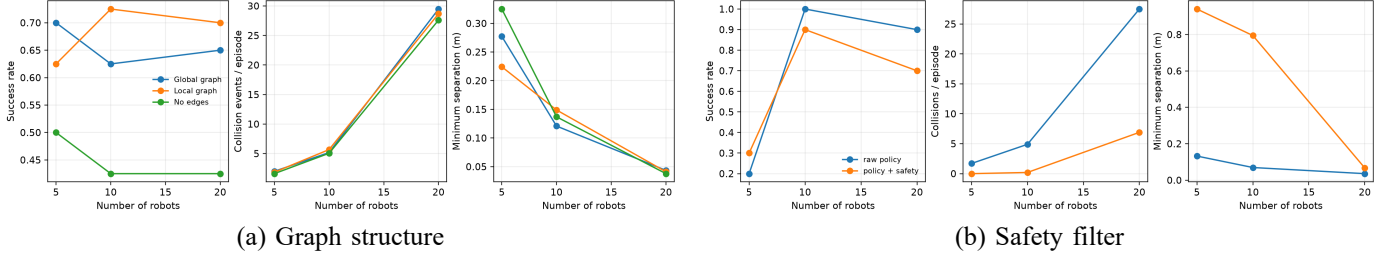


Fig. 3. Ablation results at 5, 10, and 20 robots. (a) Goal success with edge-free, local, and global graphs. (b) Goal success, safety-violation events, and minimum separation before and after safety filtering.

TABLE II

LEARNED-POLICY COMPARISON (480 EPISODES). STRICT SUCCESS REQUIRES GOAL ARRIVAL WITH NO PAIR EVER CLOSER THAN THE 0.56 M PHYSICAL DIAMETER.

Model	Success	Strict succ.	Phys. coll. events	Min. sep. (m)	Time (ms)
MLP	0.817	0.017	9.250	0.108	0.515
GNN	0.683	0.008	8.825	0.138	0.856
Neighbor-attention	0.342	0.025	8.167	0.259	1.298
Graph transformer	0.558	0.058	8.833	0.221	1.365

graph transformer achieves 0.983 and 1.000 under the same conditions. At 20 robots, goal success is 0.758 for the GNN and 0.842 for the transformer, and strict success is 0.708 and 0.767. At 30 and 40 robots, strict success decreases to 0.250 and 0.000 for the GNN, and to 0.242 and 0.017 for the transformer. At 40 robots, mean physical collision events reach 6.883 for the GNN and 9.025 for the transformer, with minimum separation near 0.4 m. GNN mean control time increases from 0.890 ms at 5 robots to 2.066 ms at 40 robots.

Without filtering, the GNN achieves goal success of 0.975–1.000 across team sizes, but strict collision-free success is 0.092 at 5 robots and 0.000 at larger sizes. The unfiltered transformer’s strict success does not exceed 0.117. Filtering therefore improves collision-free arrival, particularly at 5–10 robots, while reducing goal-reaching success. At 30–40 robots, both completion and safety deteriorate in the filtered policies, limiting their use at these densities.

G. Seed Stability

Three independently trained GNN checkpoints give mean success of 0.517 ± 0.388 , 0.767 ± 0.161 , and 0.850 ± 0.115 at 5, 10, and 20 robots. The standard deviations are computed across checkpoint-level mean success rates. Variation is largest at 5 robots, indicating sensitivity to the training seed in this setting. This analysis evaluates training variability separately

TABLE III

MULTI-SCALE TEST WITH THE SAFETY FILTER OVER 30 UNSEEN SEEDS (1800 EPISODES), AGGREGATED OVER FOUR MAP FAMILIES AND INDEXED BY ROBOT COUNT. STRICT SUCCESS REQUIRES GOAL ARRIVAL WITH NO PHYSICAL COLLISION.

Model	5	10	20	30	40
<i>Goal success / strict collision-free success</i>					
GNN	0.967/0.967	0.958/0.958	0.758/0.708	0.400/0.250	0.042/0.000
Transformer	0.983/0.983	1.000/1.000	0.842/0.767	0.467/0.242	0.108/0.017
<i>Physical collision events per episode</i>					
GNN	0.000	0.000	0.075	1.158	6.883
Transformer	0.000	0.000	0.217	2.267	9.025
<i>Minimum separation (m) / control time (ms)</i>					
GNN	0.97/0.89	0.95/1.04	0.87/1.37	0.61/1.70	0.41/2.07
Transformer	1.00/1.50	0.94/1.65	0.80/1.99	0.56/2.33	0.34/2.69

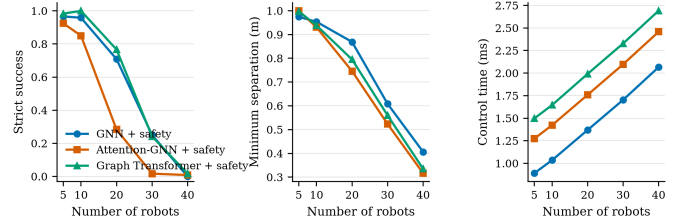
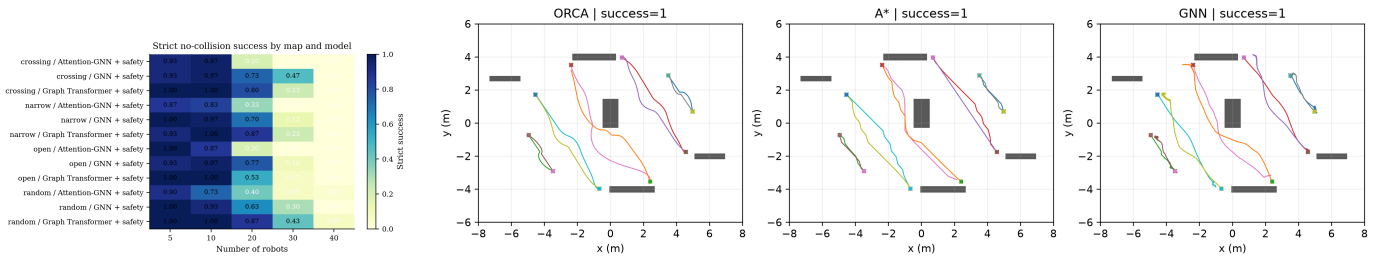


Fig. 4. Strict collision-free success, minimum separation, and mean control time with safety filtering at different team sizes. Collision-free success decreases as the number of robots exceeds 20.

from the episode-level averages reported for fixed checkpoints in the preceding comparisons.

VII. LIMITATIONS AND CONCLUSION

This paper presents a dynamic graph policy for multi-robot navigation with relative-motion edge features, a joint imitation objective, and an external safety filter. Local message passing improves goal success relative to an edge-free policy of the same capacity, while global connectivity offers no consistent additional benefit. Safety filtering increases inter-robot separation and supports high collision-free success at 5–10 robots



(a) Strict success by map and density

(b) Representative trajectories

Fig. 5. (a) Strict collision-free success by map family and population size; *random* and *narrow* layouts are the most demanding at high density. (b) Representative trajectories, with communication edges drawn as transient links between robots within the radius.

in the multi-scale test. The GNN maintains millisecond-level control time across the tested team sizes.

The experiments also identify limits. The mean-aggregation GNN does not exceed the independent MLP’s goal success in the main comparison, and strict collision-free success declines at 30–40 robots. The main GNN checkpoint is trained at one team size, and the rule-based expert constrains the behaviors available for imitation. The RVO-style baseline uses finite candidate sampling and should not be equated with a verified RVO implementation. Evaluation is limited to simulation. Further work will examine training with explicit safety constraints, comparison with reference RVO implementations, cross-map generalization, and physical-robot experiments.

DATA AVAILABILITY

The code, configuration files, and experiment records supporting this study are available from the corresponding author on reasonable request.

REFERENCES

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [2] P. Fiorini and Z. Shiller, “Motion planning in dynamic environments using velocity obstacles,” *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.
- [3] P. Long, T. Fanl, X. Liao, W. Liu, H. Zhang, and J. Pan, “Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning,” *IEEE*, 2018, pp. 6252–6259.
- [4] N. Thumiger and M. Deghat, “A multi-agent deep reinforcement learning approach for practical decentralized uav collision avoidance,” *IEEE Control Systems Letters*, vol. 6, pp. 2174–2179, 2022.
- [5] M. M. R. Komol, B. Tidd, W. Browne, F. Maire, J. Williams, and D. Howard, “Learning behaviours for decentralised multi-robot collision avoidance in constrained pathways using curriculum reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 10, no. 8, pp. 8538–8545, 2025.
- [6] P. Feng, R. Shi, S. Wang, J. Liang, X. Yu, S. Li, and W. Wu, “Safe and efficient multi-agent collision avoidance with physics-informed reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 9, no. 12, pp. 11 138–11 145, 2024.
- [7] M. Parooei, M. T. Masouleh, and A. Kalhor, “Semi-supervised deep reinforcement learning in decentralized multi-agent collision avoidance and path planning in a complex environment,” *IEEE*, 2023.
- [8] Y. Niu, F. Zhu, M. Wei, Y. Du, and P. Zhai, “A multi-ship collision avoidance algorithm using data-driven multi-agent deep reinforcement learning,” *Journal of Marine Science and Engineering*, vol. 11, no. 11, p. 2101, 2023.
- [9] Y. Wang and Y. Zhao, “Multiple ships cooperative navigation and collision avoidance using multi-agent reinforcement learning with communication,” *Ocean Engineering*, vol. 320, p. 120244, 2025.

- [10] X. Diao, W. Chi, and J. Wang, “Graph neural network based method for robot path planning,” *Biomimetic Intelligence and Robotics*, vol. 4, no. 1, p. 100147, 2024.
- [11] X. He, X. Shi, J. Hu, and Y. Wang, “Multi-robot navigation with graph attention neural network and hierarchical motion planning,” *Journal of Intelligent & Robotic Systems*, vol. 109, no. 2, 2023.
- [12] M. Goarin and G. Loianno, “Graph neural network for decentralized multi-robot goal assignment,” *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4051–4058, 2024.
- [13] Q. Yang, J. Lu, Y. Zhang, and S. Shao, “Multi-uav cooperative path planning via graph neural network and proximal policy optimization,” *IEEE Access*, vol. 13, pp. 193 575–193 588, 2025.
- [14] G. Yang, L. An, and C. Zhao, “Collision/obstacle avoidance coordination of multi-robot systems: A survey,” *Actuators*, vol. 14, no. 2, p. 85, 2025.